An RDBMS Lab Project Report on

Dermatology Clinic Management System

BACHELOR OF TECHNOLOGY

In Computer Science and Engineering

2026-2027

By

Hitesh Pundhir

Registration Number: 2502052365

Project Route: Case 2



MANIPAL UNIVERSITY
JAIPUR

Under the guidance of

Ms. Kirti Paliwal

Department of Computer Science and Engineering

School of CSE and IT

Manipal University Jaipur

Jaipur, Rajasthan

1. Introduction

This project is a small MySQL database for a dermatology clinic, meant for front-desk staff and doctors. It links patients, dermatologists, appointments and treatments, so it is quick to check who visited, which doctor they met and what the treatment cost. It follows the Case 2 ER diagram and is tested with 8 patients, 4 dermatologists, 10 appointments and 12 treatments. Main modules:

- **Patient and dermatologist records:** patient and doctor details.
- **Appointment booking:** each one links a patient to a dermatologist.
- **Treatment records:** treatment type and cost, linked to an appointment.
- **Views and stored programs:** reports, cost totals and data checks.
- **Queries on sample data:** joins, subqueries, grouping and aggregate functions.

2. Problem Statement and Scope

In a small clinic, patient, doctor and billing details often sit in separate registers, so finding one patient's visit history or total bill is slow and error-prone. This project links them through primary and foreign keys, so one query brings the data together. The aim is quicker visit look-ups and correct treatment totals.

- **Will do:** joins, subqueries, views, indexes, transactions, triggers, procedures.
- **Will not do:** user interface, online booking, or NoSQL (MySQL suits this data).

3. Tools and Technologies Used

Tool / Technology	Use in this project
MySQL 8.0+	Relational database and SQL statements.
MySQL Workbench	Writing and running SQL, and checking query results.
SQL features	DDL, DML, joins, subqueries, views, transactions and indexes.
MySQL stored programs	Procedures (one with a cursor), a function and triggers, used in place of Oracle PL/SQL.

Table 1: Software and tools used in the project.

4. Database Design

4.1 Entities and Attributes

Entity	Attributes	Primary Key
Patient	PatientID, FirstName, LastName, DOB, Phone	PatientID
Dermatologist	DoctorID, Name, Specialization, LicenseNo	DoctorID
Appointment	ApptID, Date, PatientID, DoctorID	ApptID
Treatment	TreatID, Type, Cost, ApptID	TreatID

Table 2: Entities, attributes and primary keys of the clinic database.

4.2 ER Diagram

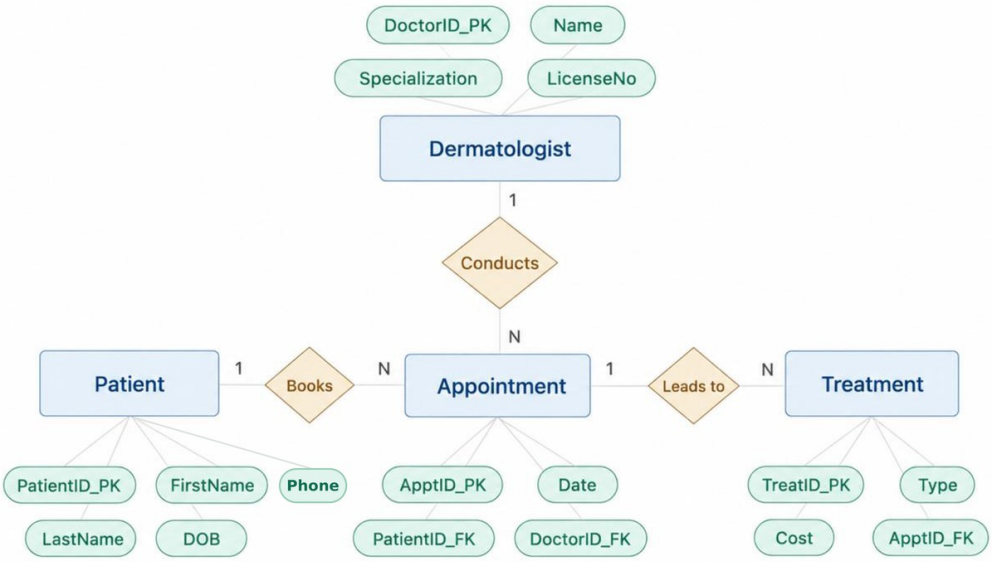


Figure 1: ER diagram of the clinic database with 1:N relationships.

4.3 Relational Schema and Constraints

Column	Data Type	Constraints
Patient		
PatientID	INT	PRIMARY KEY, AUTO_INCREMENT
FirstName, LastName	VARCHAR(50)	NOT NULL
DOB	DATE	NOT NULL, CHECK (DOB >= '1900-01-01')
Phone	VARCHAR(15)	NULL, DEFAULT NULL
Dermatologist		
DoctorID	INT	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(100)	NOT NULL
Specialization	VARCHAR(100)	NOT NULL, DEFAULT 'General Dermatology'
LicenseNo	VARCHAR(30)	NOT NULL, UNIQUE
Appointment		
ApptID	INT	PRIMARY KEY, AUTO_INCREMENT
Date	DATE	NOT NULL
PatientID	INT	NOT NULL, FOREIGN KEY → Patient(PatientID)
DoctorID	INT	NOT NULL, FOREIGN KEY → Dermatologist(DoctorID)
Treatment		
TreatID	INT	PRIMARY KEY, AUTO_INCREMENT
Type	VARCHAR(100)	NOT NULL
Cost	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00, CHECK (Cost >= 0)
ApptID	INT	NOT NULL, FOREIGN KEY → Appointment(ApptID)

Table 3: Relational schema with data types and constraints.

4.4 Relationships

- **Patient–Appointment (1:N):** Appointment.PatientID → Patient
- **Dermatologist–Appointment (1:N):** Appointment.DoctorID → Dermatologist
- **Appointment–Treatment (1:N):** Treatment.ApptID → Appointment

5. Implementation

5.1 Database Creation

```
CREATE TABLE Appointment (
   ApptID INT AUTO_INCREMENT PRIMARY KEY,
   `Date` DATE NOT NULL,
   PatientID INT NOT NULL,
   DoctorID INT NOT NULL,
   FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
      ON UPDATE CASCADE ON DELETE RESTRICT,
   FOREIGN KEY (DoctorID) REFERENCES Dermatologist(DoctorID)
      ON UPDATE CASCADE ON DELETE RESTRICT
);
```

Listing 1: CREATE TABLE statement for the Appointment table.

```
CREATE TABLE Treatment (
   TreatID INT AUTO_INCREMENT PRIMARY KEY,
   Type VARCHAR(100) NOT NULL,
   Cost DECIMAL(10,2) NOT NULL DEFAULT 0.00,
   ApptID INT NOT NULL,
   CHECK (Cost >= 0),
   FOREIGN KEY (ApptID) REFERENCES Appointment(ApptID)
      ON UPDATE CASCADE ON DELETE RESTRICT
);
```

Listing 2: CREATE TABLE statement for the Treatment table.

5.2 PL/SQL Programs (MySQL Stored Programs)

Name	Type	Purpose
sp_appointment_report	Procedure	Shows an appointment with its total.
sp_treatment_cursor_report	Procedure (cursor)	Lists and totals treatments.
sp_add_treatment	Procedure	Checks the cost, then inserts.
fn_appointment_total	Function	Returns an appointment’s total cost.
trg_appointment_date_check	Trigger	Rejects a date before birth date.
trg_treatment_cost_check	Trigger	Rejects a negative cost.

Table 4: PL/SQL programs (MySQL stored programs) used in the project.

```
CREATE FUNCTION fn_appointment_total(p_appt_id INT)
RETURNS DECIMAL(12,2)
READS SQL DATA
BEGIN
   DECLARE v_total DECIMAL(12,2);
   SELECT COALESCE(SUM(Cost),0.00) INTO v_total
   FROM Treatment WHERE ApptID = p_appt_id;
   RETURN v_total;
END$$
```

Listing 3: Function fn_appointment_total.

```
CREATE TRIGGER trg_appointment_date_check
BEFORE INSERT ON Appointment FOR EACH ROW
BEGIN
   DECLARE v_dob DATE;
   SELECT DOB INTO v_dob FROM Patient WHERE PatientID = NEW.PatientID;
   IF NEW.`Date` < v_dob THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Appointment date cannot be before date of birth';
   END IF;
END$$
```

Listing 4: Trigger trg_appointment_date_check.

5.3 Output

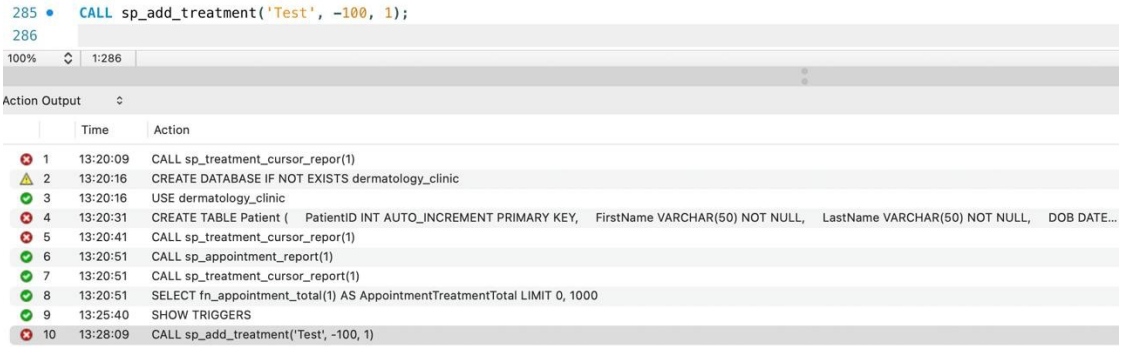


Figure 2: sp_add_treatment rejecting a negative treatment cost.

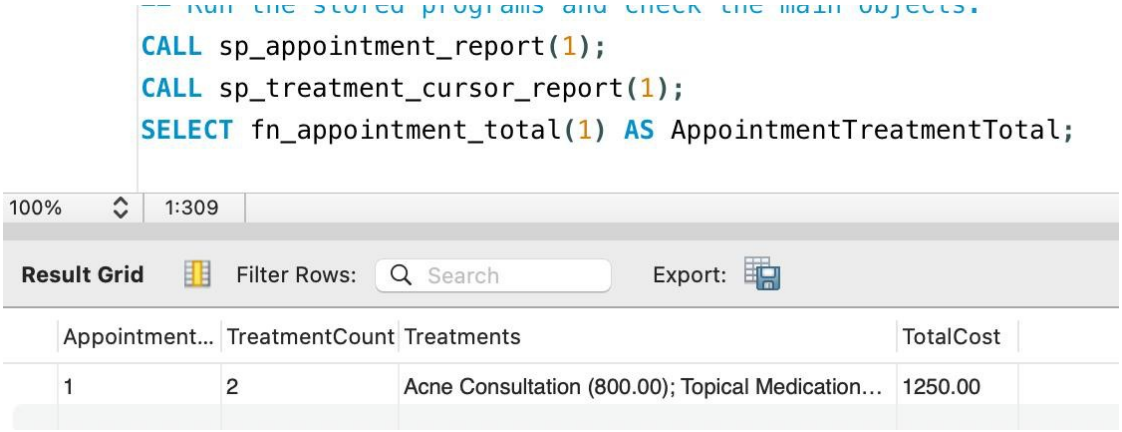


Figure 3: Program calls and cursor procedure output for appointment 1.

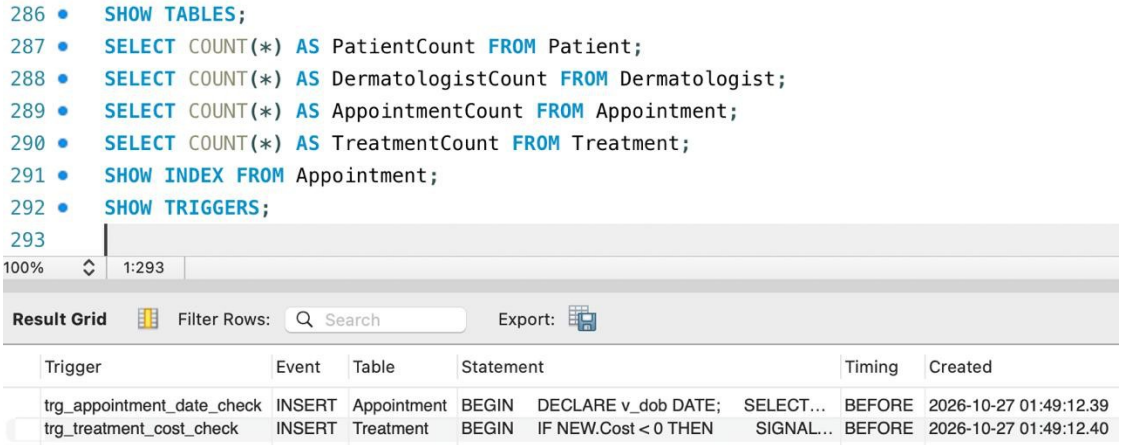


Figure 4: SHOW TRIGGERS listing the two triggers.

6. Conclusion and Future Scope

The project gave me a working database that links patients, doctors, appointments and treatments, with constraints, triggers and stored programs that keep the data correct and report treatment costs. I also practised joins, subqueries, views, indexes, transactions, a cursor and a function. Possible improvements:

- Add login and role-based access for clinic staff.
- Build a simple interface for booking appointments and updating treatments.
- Add date-wise and doctor-wise reports with proper privacy controls.

7. Bibliography / References

1. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/
2. W3Schools, SQL Tutorial. https://www.w3schools.com/sql/
3. Korth, H. F., and Sudarshan, S. Database System Concepts, 7th edition. McGraw Hill Education, 2021.
4. Beaulieu, Alan. Learning SQL, 3rd edition. O'Reilly Media, 2020.